

## Hamming Distance

## Process:

My thought process with this code was to take both words and iterate through each of their letters at the same time until the iterator was the length of the shorter word minus 1.

For the two letters, I would need to know when a bit was different between them. I used XOR to achieve this. This would leave me with 8 bits containing 0s and 1s. The amount of 1s in those bits would tell me the hamming distance between those letters, as any 1s were bits that were different between the letters.

I originally thought to use compare for finding the amount of 1s, but then took the time to understand how compare and test worked. I realized that test would actually work better for this task, as it would mask the other bits and use AND on the first bit, rather than just subtract from the byte and leave the other bits to interfere. I used test to check if the first bit was 1 as, if the zero flag wasn't flipped, then that meant that the first bit was a 1. This would mean that I needed to increase the hamming distance. After that, I would shift all the other bits to the right.

This loop would continue until the byte could be compared to 0 and flip the zero flag, meaning that there are no more 1 bits. That would mean that this letter has been completely counted, and we can go onto the next letter.

Once all the letters have been checked, the hamming distance could be printed. However, as I was testing, I found that only 1-digit numbers would print properly. In order to print the output for the second input of this assignment, I hard coded it so the ten's and one's digit would be printed one after the other. I did this by dividing the hamming distance by 10 and printing the quotient (ten's place) and the remainder (one's place) next to each other.

After printing the hamming distance, I printed out a newline and ended the program.

## The Code:

## section .data

```
newLine db " ", 0xA
lenNewLine equ $ - newLine

word1 db "this is a test"
lenWord1 equ $ - word1

word2 db "of the emergency broadcast"
lenWord2 equ $ - word2
```

## section .bss

```
ham_dis resw 2
counter resw 2
onesPlace resb 1
```

```

        tensPlace resb 1

section .text
        global _start

_start:
        mov edi, counter
        xor edi, edi

        mov esi, word1
        mov esp, word2

        mov dl, [ham_dis]
        xor dl, dl

through_each_letter:
        cmp edi, lenWord1 - 1
        jg end_program

        cmp edi, lenWord2 - 1
        jg end_program

        xor cl, cl
        xor al, al

        mov cl, [esi + edi]
        mov al, [esp + edi]

        xor cl, al

        inc edi

check_result_for_ones:
        cmp cl, 0
        jz through_each_letter

        test cl, 1
        jnz inc_ham_dis

        shr cl, 1
        jmp check_result_for_ones

end_program:
        mov [ham_dis], dl

```

```
xor eax, eax
xor cl, cl
```

```
mov ax, [ham_dis]
mov cl, 10
```

```
div cl
```

```
add ah, '0'
add al, '0'
```

```
mov [onesPlace], ah
mov [tensPlace], al
```

```
mov eax, 4
mov ebx, 1
mov ecx, tensPlace
mov edx, 1
int 0x80
```

```
mov eax, 4
mov ebx, 1
mov ecx, onesPlace
mov edx, 1
int 0x80
```

```
mov eax, 4
mov ebx, 1
mov ecx, newLine
mov edx, lenNewLine
int 0x80
```

```
mov eax, 1
int 0x80
```

```
inc_ham_dis:
    inc dl
    shr cl, 1
    jmp check_result_for_ones
```

The output:

When word1 = "foo"  
And word2 = "bar"

```
    mov eax, 4
    mov ebx, 1
    mov ecx, tensPlace
    mov edx, 1
    int 0x80

    mov eax, 4
    mov ebx, 1
    mov ecx, onesPlace
    mov edx, 1
    int 0x80

    mov eax, 4
    mov ebx, 1
    mov ecx, newLine
    mov edx, lenNewLine
    int 0x80

    mov eax, 1
    int 0x80

inc_ham_dis:
    inc dl
    shr cl, 1
    jmp check_result_for_ones
[rmekash1@linux5 assign1]$ nasm -f elf64 hamming_distance.asm
[rmekash1@linux5 assign1]$ ld hamming_distance.o
[rmekash1@linux5 assign1]$ ls
a.out  hamming_distance.asm  hamming_distance.o
[rmekash1@linux5 assign1]$ ./a.out
08
[rmekash1@linux5 assign1]$
```

When word1 = "this is a test"  
And word2 = "of the emergency broadcast"  
(I make this change manually)

```
    mov eax, 4
    mov ebx, 1
    mov ecx, onesPlace
    mov edx, 1
    int 0x80

    mov eax, 4
    mov ebx, 1
    mov ecx, newLine
    mov edx, lenNewLine
    int 0x80

    mov eax, 1
    int 0x80

inc_ham_dis:
    inc dl
    shr cl, 1
    jmp check_result_for_ones
[rmekash1@linux5 assign1]$ nasm -f elf64 hamming_distance.asm
[rmekash1@linux5 assign1]$ ld hamming_distance.o
[rmekash1@linux5 assign1]$ ls
a.out  hamming_distance.asm  hamming_distance.o
[rmekash1@linux5 assign1]$ ./a.out
08
[rmekash1@linux5 assign1]$ emacs hamming_distance.asm
[rmekash1@linux5 assign1]$ nasm -f elf64 hamming_distance.asm
[rmekash1@linux5 assign1]$ ld hamming_distance.o
[rmekash1@linux5 assign1]$ ls
a.out  hamming_distance.asm  hamming_distance.asm~  hamming_distance.o
[rmekash1@linux5 assign1]$ ./a.out
38
[rmekash1@linux5 assign1]$
```

When word1 = "hello there"  
When word2 = "how are you"

Calculated Distance: 27

0110 1000 0110 0101 0110 1100 0110 1100 0110 1111 0010 0000 0111 0100 0110 1000 0110 0101 0111 0010 0110 0101

0110 1000 0110 1111 0111 0111 0010 0000 0110 0001 0111 0010 0110 0101 0010 0000 0111 1001 0110 1111 0111 0101

0000 0000 0000 1010 0001 1011 0100 1100 0000 1110 0101 0010 0001 0001 0100 1000 0001 1100 0001 1101 0001 0000  
0 0 0 2 1 3 1 2 0 3 2 1 1 1 1 1 1 2 1 3 1 0 =

27

```

    mov eax, 4
    mov ebx, 1
    mov ecx, newLine
    mov edx, lenNewLine
    int 0x80

    mov eax, 1
    int 0x80

inc_ham_dis:
    inc dl
    shr cl, 1
    jmp check_result_for_ones
[rmekash1@linux5 assign1]$ nasm -f elf64 hamming_distance.asm
[rmekash1@linux5 assign1]$ ld hamming_distance.o
[rmekash1@linux5 assign1]$ ls
a.out  hamming_distance.asm  hamming_distance.asm~  hamming_distance.o
[rmekash1@linux5 assign1]$ ./a.out
08
[rmekash1@linux5 assign1]$ emacs hamming_distance.asm
[rmekash1@linux5 assign1]$ nasm -f elf64 hamming_distance.asm
[rmekash1@linux5 assign1]$ ld hamming_distance.o
[rmekash1@linux5 assign1]$ ls
a.out  hamming_distance.asm  hamming_distance.asm~  hamming_distance.o
[rmekash1@linux5 assign1]$ ./a.out
38
[rmekash1@linux5 assign1]$ emacs hamming_distance.asm
[rmekash1@linux5 assign1]$ nasm -f elf64 hamming_distance.asm
[rmekash1@linux5 assign1]$ ld hamming_distance.o
[rmekash1@linux5 assign1]$ ls
a.out  hamming_distance.asm  hamming_distance.asm~  hamming_distance.o
[rmekash1@linux5 assign1]$ ./a.out
27
[rmekash1@linux5 assign1]$ █

```